

Reviewer Pack — Minimal Symplectic Topological Degree and Communication Comp...

Entry 1f8ec429fa28 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 16:25:31 UTC

Minimal Symplectic Topological Degree and Communication Complexity Rank Variance

Entry ID: 1f8ec429fa28 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 16:25:31 UTC

1. Conjecture

Field A (mathematical branch): Symplectic Geometry **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For every boolean circuit with n inputs, the symplectic topological degree of its associated vector bundle is linearly correlated with the variance of its communication complexity rank, such that the variance is bounded by $O(n \log n)$. Equivalently: $\text{Var}(\text{CommRank}(\text{circuit})) = \Theta(n \log n)$, where $\text{CommRank}(\text{circuit})$ denotes the communication complexity rank of the circuit.

Rationale (proposer's reasoning):

Symplectic geometry has not been extensively applied to complexity theory, particularly in relation to communication complexity. The symplectic topological degree measures the nontriviality of the symplectic structure associated with a vector bundle, which could potentially reveal new insights into the structural properties of circuits. This conjecture aims to explore the relationship between symplectic geometry and communication complexity by connecting the geometric invariant to the rank variance, which is a fundamental measure in communication complexity.

Taxonomy category: symplectic_geometry_communication_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 07b07ef003be460b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a boolean circuit with n inputs, if the empirical variance of communication complexity ranks ($\text{Var}(\text{CommRank}(\text{circuit}))$) from 30 random seeds is within $\pm 10\%$ of the expected bound $O(n \log n)$, then support for the conjecture is provided.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.80	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from fractions import Fraction
from itertools import combinations

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        if A[i][i] == 0:
            # Find a row to swap with
            for j in range(i+1, n):
                if A[j][i] != 0:
                    A[i], A[j] = A[j], A[i]
                    break
            else:
                raise ValueError("Matrix is singular")
        factor = Fraction(A[i][i])
        for j in range(n):
            A[i][j] /= factor
        for k in range(n):
            if k != i and A[k][i] != 0:
                factor = A[k][i]
                for j in range(n):
                    A[k][j] -= factor * A[i][j]
    return A

def symplectic_topological_degree(circuit):
    # Construct the vector bundle using a constructive mapping
    n = len(circuit)
    A = [[0]*n for _ in range(n)]
    for i, j in combinations(range(n), 2):
        if circuit[i] == circuit[j]:
            A[i][j] = 1
    rank = len(gaussian_elimination(A))
```

```

    return rank

def communication_complexity_rank(circuit):
    # Placeholder function for actual computation of communication complexity rank
    return random.randint(1, n)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    metrics = []
    instances_tested = 0
    n_max = 0

    for n in n_values:
        for _ in range(5): # Sample 5 instances per size
            circuit = [random.choice([0, 1]) for _ in range(n)]
            degree = symplectic_topological_degree(circuit)
            rank = communication_complexity_rank(circuit)
            metrics.append((degree, rank))
            instances_tested += 1
            n_max = max(n_max, n)

    if not metrics:
        return {
            "metric_name": "Var(CommRank)",
            "metric_value": None,
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": "No circuits generated"
        }

    expected_bound = [n * Fraction(n).log2() for n, _ in metrics]
    empirical_variance = sum((rank - expected) ** 2 for _, rank, expected in zip(metrics, expected_bound, expected_bound))
    conjecture_holds = abs(empirical_variance - expected_bound[0]) <= 0.1 * expected_bound[0]

    return {
        "metric_name": "Var(CommRank)",
        "metric_value": empirical_variance,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{'seed': {seed}, **trial_result}}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results if result["metric_value"] is not None)

```

```

std_value = (sum((result["metric_value"] - mean_value) ** 2 for result in results if result["metric_value"] > 0)) / len(results)
support_fraction = sum(result["conjecture_holds"] for result in results) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next((seed for seed, result in zip(seeds, results) if not result["conjecture_holds"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e2dc6029.py", line 97, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e2dc6029.py", line 62, in run_trial
    degree = symplectic_topological_degree(circuit)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e2dc6029.py", line 45, in symplectic_topological_degree
    rank = len(gaussian_elimination(A))
            ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e2dc6029.py", line 27, in gaussian_elimination
    raise ValueError("Matrix is singular")
ValueError: Matrix is singular

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution before producing data to evaluate the conjecture.
 | next: Investigate and fix the error in the test code that caused it to crash, then rerun the test with a valid circuit to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14033
2	propose	ollama_remote	glm4:latest	0	0	12223
3	propose	ollama_remote	glm4:latest	0	0	13207
4	propose	ollama_remote	glm4:latest	0	0	13258
5	preregistration	ollama_remote	glm4:latest	0	0	14015
6	novelty	ollama_remote	glm4:latest	0	0	8910
7	novelty	ollama_remote	glm4:latest	0	0	8665
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25440
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11804
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15685
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11875
12	judge	ollama_remote	glm4:latest	0	0	12020

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 161135 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in *research/audit/1f8ec429fa28.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/1f8ec429fa28.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/1f8ec429fa28.tar.gz* (if generated)